

JALRAKSHA NAGPUR — MASTER PLAN & PROGRESS TRACKER

Vikshit Nagpur Hackathon — Smart City Track

How to use this file: This is your single source of truth. Paste this whole document (or the relevant section) into any LLM at any point to resume work with full context. Update the checkboxes under **PROGRESS TRACKER** as you complete steps — that section alone tells any AI assistant exactly where you are.

⚠ IMPORTANT NOTE ON THE HACKATHON ITSELF

I searched specifically for an event called "**Vikshit Nagpur Hackathon**" and could not find an official page, registration portal, or organizer listing for an event by that exact name. What I *did* find are several real, currently active "Viksit Bharat"-themed hackathons with a Nagpur/Vidarbha connection (e.g., Sankalp Bharat Hackathon 2026 at St. Vincent Pallotti College of Engineering & Technology, Nagpur — theme "Innovating Sustainable Solutions for Viksit Bharat"). It's possible: "Vikshit Nagpur Hackathon" is a smaller/college-level or very recently announced event not yet indexed by search engines, or It's a regional variant/spelling of one of these Viksit Bharat events.

Action item for you: Get the official registration link, rulebook, and exact problem-statement wording for "Smart City track" directly from the organizers (WhatsApp group / Unstop / college notice) and paste the exact problem statement text into this doc. Everything below is built to be a strong, generic **Smart City — Urban Flood & Drainage Intelligence** submission that will fit almost any reasonable version of that track's judging criteria (innovation, feasibility, technical depth, real-world impact, use of open government data).

PROGRESS TRACKER (update this as you go)

Phase 0 — Setup

- ☐ Get official hackathon problem statement / rubric / submission format
- ☐ Confirm team size, tech constraints, allowed APIs/services
- ☐ Set up GitHub repo (`(jalraksha-nagpur)`) with README, `/data`, `/backend`, `/frontend`, `/scripts`
- ☐ Set up MongoDB Atlas free cluster (or local MongoDB + PostGIS if time allows)

Phase 1 — Data Collection

- ☐ Download NMC Disaster Management Plan 2022-23 PDF (nullah/flood point data)
- ☐ Download DDMA Nagpur District Disaster Management Plan PDF
- ☐ Download NEERI Environmental Status Report Nagpur (latest year available)
- ☐ Extract OSM waterways/drains for Nagpur via Overpass Turbo → GeoJSON
- ☐ Download SRTM DEM for Nagpur (30m) from USGS EarthExplorer or OpenTopography
- ☐ Get Bhuvan LULC / Flood Hazard Zonation layer for Maharashtra/Nagpur via WMS or NDEM
- ☐ Get IMD/RMC Nagpur historical daily rainfall (as far back as accessible)
- ☐ Digitize/geo-reference the 127 historical flood/submergence points (from Gaurkhede & Adane 2023 paper, sourced from NMC Disaster Management Cell) — **read data-use note below**
- ☐ Compile 2023 & 2024 flood event list (dates, rainfall mm, areas affected, deaths) from news + Wikipedia + Groklopedia sources found

Phase 2 — Data Processing

- ☐ Clean & reproject all layers to EPSG:4326 in QGIS
- ☐ Convert SHP/KML → GeoJSON
- ☐ Build synthetic/estimated drain network (see Section D — Data Gaps) since NMC has not published a public drain-line dataset
- ☐ Compute slope, TWI from DEM in QGIS
- ☐ Assign each historical flood point to nearest drain/catchment

Phase 3 — Backend

- ☐ Design MongoDB schema (Section G)
- ☐ Build Express REST API (Section G)
- ☐ Implement bounding-box / tile-based data loading (not full dataset)
- ☐ Implement flood-risk scoring endpoint (Section H)

Phase 4 — Frontend

- ☐ React + Leaflet base map (Nagpur centered: 21.1458° N, 79.0882° E)
- ☐ Drain/nullah layer rendering with color-coded risk
- ☐ Click-to-inspect drain metadata popup
- ☐ Flood hotspot heatmap / choropleth
- ☐ Citizen flood report form (photo + GPS + depth)

Phase 5 — Flood Intelligence

- ☐ Frequency-ratio or weighted-overlay risk model (replicate Gaurkhede & Adane methodology as baseline — it's peer-reviewed and Nagpur-specific)
- ☐ "Why did this area flood" explainer using parameter breakdown
- ☐ Drainage graph (NetworkX) for upstream/downstream bottleneck analysis

Phase 6 — CivicProof (Blockchain layer)

- ☐ Decide scope: MVP = hash-anchoring only (recommended), stretch = smart contract work-order flow
- ☐ Set up Algorand TestNet account + AlgoKit
- ☐ Build hash-anchoring script (photo/GIS file → SHA-256 → on-chain note field)
- ☐ Build simple verification UI ("verify this maintenance record")

Phase 7 — Polish & Pitch

- ☐ Record demo video / prepare live demo flow
- ☐ Build slide deck: Problem → Data → Architecture → Demo → Impact → Roadmap
- ☐ Prepare "data gaps & how we'd close them" slide (judges respect honesty about real Indian civic-data limitations)
- ☐ Rehearse 3–5 min pitch

DATA HONESTY NOTE (read this before building)

Nagpur does **not** currently have a public, downloadable, parcel-level drain/stormwater network dataset (open shapefile/GeoJSON of every drain line, manhole, culvert). What exists:

NMC/NSSCDCL has done **internal GIS mapping** of ~3,216 km of sewer lines and 1.78 lakh chambers (2023–24), but this is an internal asset-management system, not published open data.

NMC is building a broader GIS civic-asset platform (2026, in progress) — also not yet public.

The most detailed **published** Nagpur drainage engineering document is the **Stormwater Drainage Master Plan / DPR (DRA Consultants, for NMC, "Master Plan for Drainage System and Rejuvenation of Lakes**

and Rivers of Nagpur City, 2031") — but this is a planning/consultancy report, not raw GIS data, and full volumes are not confirmed to be freely downloadable (fragments appear on document-sharing sites like Scribd, which you should NOT scrape/republish due to copyright — cite it, don't redistribute it).

The best **real, usable, geolocated flood dataset** for Nagpur is the set of **127 submergence points (2008–2022) along 8 water channels**, compiled by NMC's Disaster Management Cell and used in the peer-reviewed paper by Gaurkhede & Adane (2023, VNIT Nagpur). The paper itself doesn't republish the raw point coordinates as an open dataset — you would need to either (a) email the NMC Disaster Management Cell / authors for the underlying data, or (b) digitize the flood-point figure from the paper manually in QGIS as a best-effort approximation (clearly label this **ESTIMATED**, not VERIFIED).

Conclusion for your build: Be upfront in your pitch that you are combining (1) real open satellite/DEM/rainfall data, (2) real but non-GIS engineering reports, and (3) a synthetic/estimated drain network generated from OSM waterways + DEM flow accumulation, because that is the honest and technically correct state of open data for Nagpur today. This is *itself* a good hackathon narrative: "Nagpur has no public drainage GIS layer — here's how we built one from open satellite + hydrology data, and here's the civic-tech roadmap to make it official."

A. BEST NAGPUR DRAINAGE DATASETS FOUND

Name	Organization	URL
NMC Disaster Management Plan 2022-23	Nagpur Municipal Corporation	https://www.nmcnagpur.gov.in/assets/250/2022/07/mediafiles/Aapat
NEERI Environmental Status Report Nagpur	NEERI (for NMC)	https://www.nmcnagpur.gov.in/assets/250/2021/06/mediafiles/Final_2
DDMA Nagpur District Disaster Management Plan	District Disaster Management Authority, Nagpur	https://static.s3waas.gov.in/s3d1f491a404d6854880943e5c3cd9ca25/
Stormwater Drainage Master Plan / DPR (Vol. I, North Zone etc.)	NMC / DRA Consultants Ltd.	Referenced at https://www.dra.net.in/project/preparation-of-master-drainage-system-and-rejuvenation-of-lakes-and-rivers-of-nagpur-city
OpenStreetMap waterways/drains (Nagpur)	OpenStreetMap contributors	https://overpass-turbo.eu/ (query against <code>waterway=drain/stream/ditch</code>)

Name	Organization	URL
Historical flood/submergence points (127 pts, 2008-2022)	NMC Disaster Management Cell, via Gaurkhede & Adane (2023)	https://aloki.hu/pdf/2103_23412361.pdf
Flood extent/hazard layers (NDEM/Bhuvan)	NRSC/ISRO	https://ndem.nrsc.gov.in/ (GeoServer: <code>ndem.nrsc.gov.in/geoserver_v3</code>) app1.nrsc.gov.in/disaster/disaster.php?id=flood_hz

B. BEST GOVERNMENT PORTALS / RESOURCES

Portal	What it's for	URL
Nagpur Municipal Corporation (official site)	Assets, reports, disaster plans, zone offices	https://www.nmcnagpur.gov.in
District Nagpur (NIC) portal	Municipal corporation info, contacts	https://nagpur.gov.in
MRSAC (Maharashtra Remote Sensing Applications Centre)	Headquartered in Nagpur itself (VNIT Campus) — state RS/GIS body; runs Geoportal with drought, water, agriculture layers	https://mrsac.gov.in/MRSAC/ and Geoportal at https://mrsac.gov.in/MRSAC/pages/web_applications
Bhuvan (ISRO/NRSC)	LULC, geomorphology, flood hazard, DEM thematic layers via WMS/WMTS	https://bhuvan.nrsc.gov.in
Bhuvan WMS how-to	Technical guide to consuming Bhuvan layers in QGIS/OpenLayers	https://bhuvan.nrsc.gov.in/wiki/index.php/How_to_use_WMS_service
NDEM (National Database for Emergency Management)	Flood hazard zonation maps, GeoServer-based, extractable	https://ndem.nrsc.gov.in
India-WRIS	Rainfall, river discharge, water bodies, basin/watershed layers, reservoirs	https://indiawris.gov.in

Portal	What it's for	URL
IMD / RMC Nagpur	Regional Meteorological Centre Nagpur — local rainfall data	https://www.imdnagpur.gov.in/pages/rainfall_drms.php
IMD Pune CDSP	Gridded temperature & rainfall download	https://cdsp.imdpune.gov.in
IMD API Management portal	City forecasts, nowcast, rainfall, warnings via API key	https://api.imd.gov.in/public/index.php
data.gov.in (OGD Platform)	Rainfall catalogues (CWC/IMD/NRSC gridded + station data)	https://www.data.gov.in/catalog/rainfall
USGS EarthExplorer	SRTM DEM (30m), Landsat imagery	https://earthexplorer.usgs.gov
OpenTopography	Easier DEM download (SRTM, sometimes higher-res)	https://opentopography.org
OpenStreetMap / Overpass Turbo	Roads, waterways, buildings, land use	https://overpass-turbo.eu
CGWB (Central Ground Water Board)	Lithology, aquifer maps	https://cgwb.gov.in
CPCB	Water quality, drain water quality monitoring	https://cpcb.nic.in/water-quality-monitoring-of-drain/

C. BEST ENVIRONMENTAL / RAINFALL / FLOOD DATASETS

Parameter	Best Source	Notes
DEM / Elevation	USGS SRTM 30m via EarthExplorer or OpenTopography	Used in the VNIT Nagpur flood-susceptibility proven workable for this exact city
Slope, TWI	Derived from SRTM DEM in QGIS (Terrain Analysis / SAGA tools)	DERIVED, not a raw dataset
LULC	Bhuvan NRSC 1:50,000 LULC thematic layer, or manual digitization from Google Earth imagery (method used in the VNIT paper)	Bhuvan via WMS; NRSC LULC doc: https://bhuvan.nrsc.gov.in/2dresources/thematic/2LULC
Soil	FAO UN Soils Portal (global, 1:50,000-ish)	Nagpur is ~91.5% clayey soil per the VNIT study on infiltration, relevant to your risk model
Buildings	OpenStreetMap <code>building=*</code> tag export via Overpass Turbo	Incomplete but usable base
Roads	OpenStreetMap <code>highway=*</code>	Good coverage in Indian metro areas generally
Water bodies / lakes	Bhuvan water body thematic layer (https://bhuvan.nrsc.gov.in/thematic/thematic/index.php) + OSM <code>natural=water</code>	Cross-check both; Nagpur has documented - shrinkage of water bodies (per Grokipedia su academic sources) — a strong narrative data
Historical flood locations	Gaurkhede & Adane 2023 paper (127 points, 2008-2022)	See data-honesty note above
Historical rainfall	RMC Nagpur (IMD) + CDSP IMD Pune gridded data + India-WRIS rainfall dashboard	Nagpur avg annual rainfall ≈ 1205 mm (DDMA 2013 saw the historic max 5-day rainfall of 333 mm caused major flooding)
Real-time/forecast rainfall	IMD API (registration required)	For "flood-aware routing" / live features
Flood extent from satellite	NDEM/Bhuvan flood hazard + event layers; also check Sentinel-1 SAR via Copernicus Open Access Hub for near-real-time flood mapping of the Sept 2023 event	SAR (radar) can see through monsoon clouds good technical talking point

Documented historical flood events (for your "why did this flood" narrative + citizen trust):

23–24 Sept 2023 — 109–145 mm rainfall (sources vary), Nag River overflowed, 4 deaths, 400+ evacuated, ~10,000 houses affected. Contributing factors per multiple sources: inadequate stormwater system, encroachment on Nag River, water-hyacinth (*Eichhornia*) blockage at Ambazari Lake outfalls.

July 2013 — historic max rainfall event (used as the design storm in the VNIT susceptibility study).

20 July 2024 — heavy rains, waterlogging, disrupted traffic across Nagpur/Vidarbha.

Post-2023-flood government response: ₹266.63 crore integrated drainage plan approved Nov 2023; additional ₹204.72 crore sanctioned Feb 2024 for nullah desilting, road repairs, and 5 new pumping stations — **as of Sept 2025, only ₹86.63 crore released, 1 pumping station operational, retaining-wall work stalled**. This is a *very strong* hackathon talking point: your platform can track exactly this kind of implementation gap using the CivicProof work-order layer.

D. DATA GAPS (be explicit about these — judges respect this)

Gap	Why it matters	Workaround for hackathon MVP
No public drain-line/manhole GIS layer	Core to your whole product idea	Derive a synthetic drain network from OSM waterways + DEM-derived flow accumulation/streamlines in QGIS (<code>r.watershed</code> in GRASS or ArcGIS Hydrology toolset). Label as ESTIMATED .
No public drain capacity/condition/blockage records	Needed for real risk scoring	Use ESTIMATED capacity from pipe/channel type + width assumptions from the Master Plan DPR methodology (rational method); simulate blockage % for demo purposes, clearly labeled as demo/synthetic data with a note on how real IoT sensors or citizen reports would replace it
No raw coordinates for the 127 historical flood points	Ground-truth for validation	Digitize approximate points from Fig. 3 of the Gaurkhede & Adane paper by georeferencing the published map image in QGIS (best-effort, label ESTIMATED); simultaneously email NMC Disaster Management Cell / VNIT authors requesting the dataset (include this ask in your pitch as future work)
Full stormwater Master Plan DPR volumes not confirmed freely downloadable	The actual engineering answer to "why does X area flood"	Cite it; consider an RTI (Right to Information) request to NMC Public Health Engineering Dept for the DPR, and mention this as your data-acquisition roadmap in the pitch
No live sensor network	Real-time water level / blockage detection	Mock a small IoT layer (Node.js script emitting simulated MQTT/HTTP sensor payloads) — a very common, judge-accepted hackathon pattern
No official current flood-prone area shapefile	For flood_zones layer in your DB	Recreate approximate zones using the frequency ratio flood susceptibility methodology from the VNIT Nagpur paper — it's peer-reviewed, replicable, and Nagpur-specific. This is your strongest, most defensible "real" flood-risk basis.

E. HOW TO OBTAIN THE DATA YOU CAN'T FIND ONLINE

RTI (Right to Information) request to NMC Public Health Engineering Department for the Stormwater Drainage Master Plan / DPR volumes and any GIS shapefiles from the NSSCDCL sewer-mapping project (3,216 km network, 1.78 lakh chambers). Mention this as a "next step" in your pitch — judges like seeing a real-world data-acquisition plan, not just "we'll get more data somehow."

Email/contact NMC Disaster Management Cell (referenced as data source "DMC-NMC 2022" in the VNIT paper) for the raw 127 flood-point coordinates.

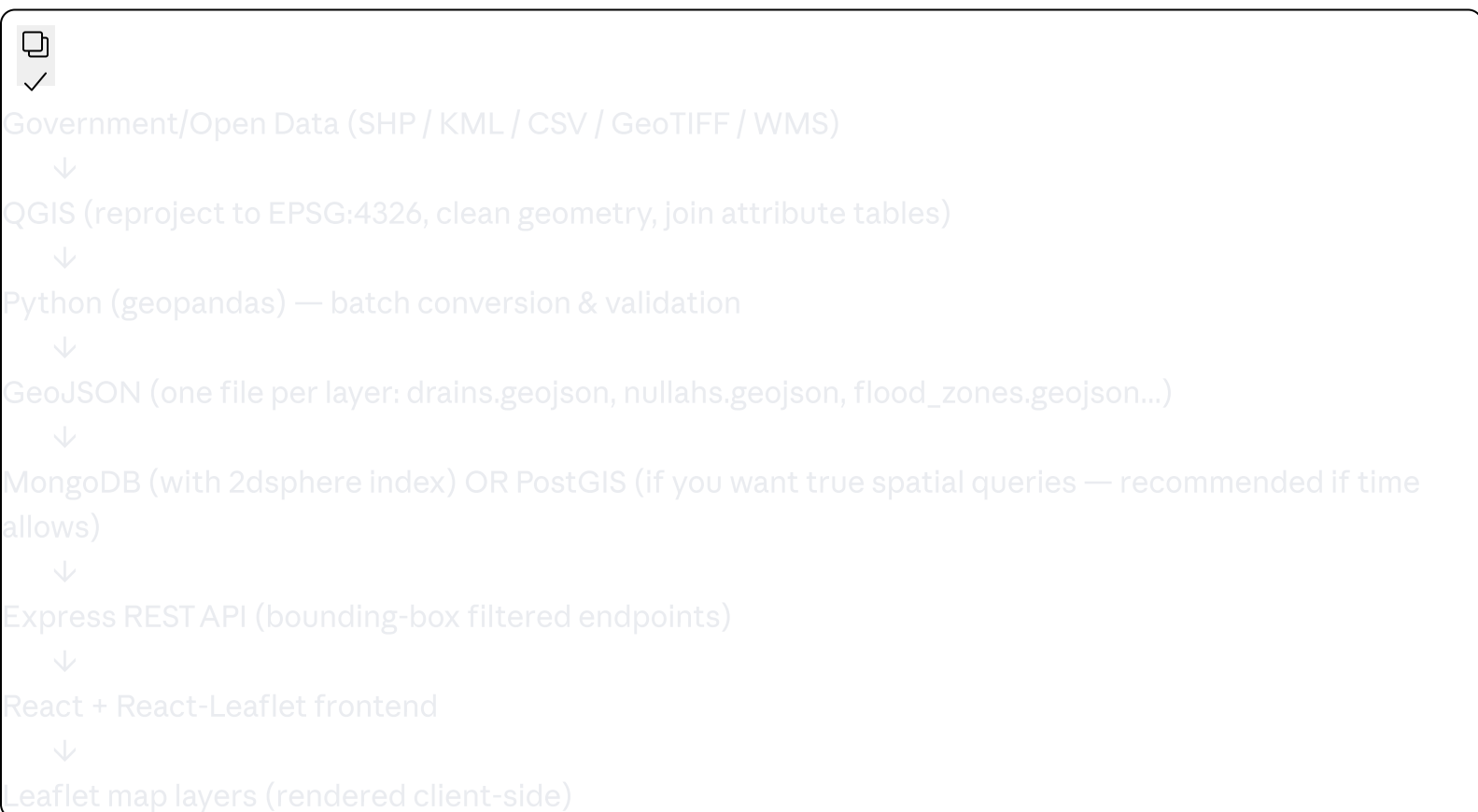
Contact the VNIT Nagpur authors (Gaurkhede, N.T. and Adane, V.S., Dept. of Architecture and Planning, VNIT Nagpur — corresponding email in the paper: namrata2707@gmail.com) — academic authors are often willing to share underlying data or point you to it for genuine research/civic-tech use.

NMC GIS platform (in development, 2026) — once launched, citizens are expected to get city-level data access per the news coverage found; worth checking nmcnagpur.gov.in periodically or reaching out to ask about hackathon/developer access.

MRSAC (Nagpur-based!) — since MRSAC is physically headquartered in Nagpur at VNIT campus, a direct visit/email inquiry is realistic and could yield local geospatial layers not indexed online.

Crowdsource via your own app — citizen flood reports become your own growing verified dataset over time; frame this explicitly as part of your product's data strategy, not just a "feature."

F. GIS → GEOJSON → DATABASE → API → LEAFLET PIPELINE



Converting formats



bash

Shapefile → GeoJSON

```
ogr2ogr -f GeoJSON -t_srs EPSG:4326 drains.geojson drains.shp
```

KML → GeoJSON

```
ogr2ogr -f GeoJSON -t_srs EPSG:4326 nullahs.geojson nullahs.kml
```

CSV with lat/lon → GeoJSON (using geopandas)

```
python3 -c "
```

```
import geopandas as gpd
```

```
import pandas as pd
```

```
df = pd.read_csv('flood_points.csv')
```

```
gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df.lon, df.lat), crs='EPSG:4326')
```

```
gdf.to_file('flood_points.geojson', driver='GeoJSON')
```

```
"
```

Example GeoJSON — a single drain feature



json

```
{
  "type": "Feature",
  "properties": {
    "drain_id": "D042",
    "type": "secondary",
    "capacity_m3_hr": 8000,
    "condition": "poor",
    "blockage_pct": 72,
    "risk_score": 87,
    "data_confidence": "estimated"
  },
  "geometry": {
    "type": "LineString",
    "coordinates": [
      [79.0862, 21.1465],
      [79.0871, 21.1459],
      [79.0879, 21.1451]
    ]
  }
}
```

Loading into MongoDB with geospatial index



javascript

// One-time import script

```
const { MongoClient } = require("mongodb");
const fs = require("fs");

async function importDrains() {
  const client = await MongoClient.connect(process.env.MONGO_URI);
  const db = client.db("jalraksha");
  const geojson = JSON.parse(fs.readFileSync("drains.geojson"));

  const docs = geojson.features.map(f => ({
    ...f.properties,
    geometry: f.geometry
  }));

  await db.collection("drains").insertMany(docs);
  await db.collection("drains").createIndex({ geometry: "2dsphere" });
  await client.close();
}

importDrains();
```

Leaflet rendering (React-Leaflet)



jsx

```
import { MapContainer, TileLayer, GeoJSON, Popup } from "react-leaflet";
```

```
function riskColor(score) {  
  if (score >= 80) return "#d32f2f"; // red - critical  
  if (score >= 60) return "#f57c00"; // orange - high  
  if (score >= 40) return "#fbc02d"; // yellow - moderate  
  return "#388e3c"; // green - low  
}
```

```
function DrainLayer({ drains }) {  
  return (  
    <GeoJSON  
      data={drains}  
      style={({ feature }) => ({  
        color: riskColor(feature.properties.risk_score),  
        weight: 3  
      })}  
      onEachFeature={({ feature, layer }) => {  
        layer.bindPopup(`  
          <b>Drain ${feature.properties.drain_id}</b><br/>  
          Capacity: ${feature.properties.capacity_m3_hr} m³/hr<br/>  
          Condition: ${feature.properties.condition}<br/>  
          Blockage: ${feature.properties.blockage_pct}%<br/>  
          Risk Score: ${feature.properties.risk_score}/100  
        `);  
      })  
    </GeoJSON>  
  );  
}
```

```
export default function App() {  
  return (  
    <MapContainer center={[21.1458, 79.0882]} zoom={12} style={{ height: "100vh" }}>  
      <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />  
      { /* Load drains via fetch to your API, then <DrainLayer drains={data}/> */ }  
    </MapContainer>  
  );  
}
```

Different layer types on Leaflet:

Drain/nullah lines → `GeoJSON` with `LineString`, colored by risk

Flood polygons → `GeoJSON` with `Polygon`/`MultiPolygon`, semi-transparent fill (`fillOpacity: 0.4`)

Flood points / citizen reports → `CircleMarker` or clustered markers (`react-leaflet-cluster`)

Catchments → `Polygon` outlines, toggleable layer

Rainfall stations → custom icon `Marker` with popup showing latest mm reading

Critical infrastructure (hospitals, pumping stations) → distinct icon set, always-on-top layer

G. DATABASE / API DESIGN

MongoDB Collections (schema sketch)



javascript

```
// drains
{
  _id, drain_id, name, type: "major|secondary|minor",
  geometry: { type: "LineString", coordinates: [...] },
  capacity_m3_hr, width_m, depth_m, material,
  condition: "good|fair|poor",
  blockage_pct, last_desilted_date,
  catchment_id, downstream_drain_id,
  risk_score, data_confidence: "verified|derived|estimated"
}

// nullahs
{
  _id, name, geometry: { type: "LineString"|"MultiLineString", coordinates: [...] },
  order (stream order), avg_width_m, encroachment_flag,
  outfall_to: "Nag River" | "Pili River" | "Pora River"
}

// culverts
{ _id, name, location: { type: "Point", coordinates: [...] },
  span_m, condition, associated_drain_id, associated_road }

// catchments
{ _id, name, geometry: { type: "Polygon", coordinates: [...] },
  area_km2, imperviousness_pct, avg_slope_deg, outfall_drain_id }

// flood_zones
{ _id, name, geometry: { type: "Polygon", coordinates: [...] },
  susceptibility_class: "very_high|high|moderate|low|very_low",
  fsi_score, method: "frequency_ratio_2023_vnit_study|custom_model" }

// flood_events
{ _id, date, rainfall_mm, duration_hrs, areas_affected: [String],
  deaths, houses_affected, source_url, verified_by: "news|govt_report" }

// rainfall
{ _id, station_name, location: { type: "Point", coordinates: [...] },
  date, rainfall_mm, source: "IMD|RMC_Nagpur|India-WRIS" }

// citizen_reports
{ _id, user_id (or anonymous_token), location: { type: "Point", coordinates: [...] },
  timestamp, photo_url, estimated_depth_cm, description, status: "pending|verified|resolved" }

// critical_infrastructure
```



```
{_id, name, type: "hospital|school|pumping_station|bridge",
location: { type: "Point", coordinates: [...] }, flood_risk_flag }

// interventions
{_id, drain_id, work_order_id, type: "desilting|repair|widening",
status: "planned|in_progress|completed", cost_estimate,
before_photo_url, after_photo_url, blockchain_hash, blockchain_tx_id }

// sensors
{_id, sensor_id, type: "water_level|rainfall|flow|blockage",
location: { type: "Point", coordinates: [...] },
latest_reading, latest_reading_time, status: "active|offline" }
```

API Endpoints

```
GET /api/drains?bbox=79.05,21.10,79.15,21.20&zoom=13 → returns only drains in visible map bounds
GET /api/drains/:id → single drain full metadata
GET /api/drains/nearby?lat=..&lng=..&radius_m=500
GET /api/drains/high-risk?threshold=70
GET /api/flood-zones?bbox=...
GET /api/flood-events?from=2020-01-01&to=2024-12-31
GET /api/rainfall/latest?station=Nagpur
POST /api/citizen-reports → submit a flood report
GET /api/citizen-reports?bbox=...&status=verified
GET /api/interventions/:drainId
POST /api/interventions/:id/verify → triggers CivicProof hash anchoring
```

Avoiding sending the whole city to the browser

Bounding-box queries: every GET endpoint requires a `bbox` param; MongoDB `2dsphere` index + `$geoWithin`/`$geoIntersects` filters server-side.

Zoom-based simplification: at low zoom, serve simplified geometry (precompute simplified versions with `turf.simplify` or `mapshaper`) or just points/clusters instead of full line detail; at high zoom, serve full-resolution geometry.

Tile-based approach (better, if time allows): pre-generate **vector tiles** (`.mbtiles` via `tippecanoe`) and serve via `Leaflet.VectorGrid` — this is the production-grade solution and a great “we thought about scale” talking point for judges.

Pagination + debounced map-move events: only refetch when the user stops panning/zooming (`moveend` event in Leaflet), not on every pixel of movement.

H. FLOOD-RISK INTELLIGENCE ARCHITECTURE

Recommended baseline: replicate the VNIT Nagpur Frequency Ratio (FR) model

This is the **only peer-reviewed, Nagpur-specific flood model with published parameter weights** you can point to. Use it as your core scoring logic, then layer drain-level data on top for a hyperlocal score.

City-level parameters (from the VNIT study) — use as base weights:

Parameter	Max FR found	Interpretation
Surface runoff	4.30	Strongest predictor
Landform (water bodies)	3.58	Strong
Lithology (Lameta formation)	2.36	Strong
Rainfall (1218-1226mm zone)	1.82	Moderate-strong
Elevation (292-301m)	1.57	Moderate
LULC (built-up)	1.24	Moderate
Slope, distance-from-river, TWI ~1.1-1.16		Weak-moderate
Soil texture	~1.0-1.06	Weak

Drain-level score (your original contribution — combine with the above):



```
risk_score = w1*(rainfall_intensity_norm)
            + w2*(1 - elevation_norm)    // lower elevation = higher risk
            + w3*(1 - slope_norm)        // flatter = higher risk
            + w4*(1 - capacity_norm)      // lower capacity = higher risk
            + w5*(blockage_pct / 100)
            + w6*(condition_penalty)      // poor=1, fair=0.5, good=0
            + w7*(historical_flood_proximity) // near a known flood point = higher risk

// Normalize to 0-100
```

Example output (matches your original prompt's format):



```
Drain D042
Capacity: 8000 m³/hr
Blockage: 72%
Condition: Poor
Rainfall: High
Elevation: Low
Risk Score: 87/100 ●
Recommended: Urgent cleaning + downstream inspection
```

Identifying flood hotspots

Cluster high `risk_score` drains spatially (DBSCAN via `scikit-learn` in a small Python microservice, or just grid-based aggregation) → hotspot polygons.

Identifying drainage bottlenecks

A drain segment is a **bottleneck** if: its downstream capacity < upstream inflow (sum of contributing catchment runoff), OR blockage_pct is high on a high-order (major) drain.

"Why did this area flood?" explainer

Show a breakdown card: rainfall on that day + elevation + nearest drain's blockage/condition + whether it's within X meters of a `flood_zones` very-high-susceptibility polygon + any recent `citizen_reports` nearby.

This narrative breakdown (not just a number) is what impresses hackathon judges.

Which drain to fix first

Rank by: `risk_score × population_served_estimate × (1/cost_estimate)` — i.e., prioritize high-risk, high-impact, low-cost interventions first (classic public-infrastructure prioritization logic, easy to explain on stage).

Leaflet visualization

Drains: line color red/orange/yellow/green by `risk_score` bucket (as coded above)

Flood zones: filled polygon, opacity by susceptibility class

Hotspot clusters: pulsing/animated circle markers at high zoom-out

Legend component fixed to map corner

I. DRAINAGE NETWORK AS A GRAPH

Represent the hierarchy `Minor → Secondary → Primary → Nullah → River/Outfall` as a **directed graph** using Python + NetworkX.

✓

```
python
```

```
import networkx as nx
```

```
G = nx.DiGraph()
```

```
# Add nodes (drains/nullahs) with attributes
```

```
G.add_node("D042", type="secondary", capacity=8000, blockage=0.72)
```

```
G.add_node("N01_NagRiver", type="nullah", capacity=50000, blockage=0.10)
```

```
# Add edges = flow direction (upstream -> downstream)
```

```
G.add_edge("D042", "N01_NagRiver", flow_direction="downstream")
```

```
# Find all upstream drains feeding a given point (e.g., before recommending a fix)
```

```
upstream = nx.ancestors(G, "N01_NagRiver")
```

```
# Find bottlenecks: nodes where in-degree flow > out-degree capacity
```

```
def find_bottlenecks(G):
```

```
    bottlenecks = []
```

```
    for node in G.nodes:
```

```
        inflow = sum(G.nodes[p]["capacity"] * (1 - G.nodes[p]["blockage"]) for p in G.predecessors(node))
```

```
        outflow_capacity = G.nodes[node]["capacity"] * (1 - G.nodes[node]["blockage"])
```

```
        if inflow > outflow_capacity:
```

```
            bottlenecks.append(node)
```

```
    return bottlenecks
```

```
# Areas affected by a blocked drain = everything downstream of it
```

```
def downstream_impact(G, drain_id):
```

```
    return nx.descendants(G, drain_id)
```

This lets you answer, live in your demo: *"If Drain D042 fails completely, which downstream areas/nullahs are affected?"* — a genuinely useful and demo-friendly feature.

Building the graph itself: derive edges from your DEM flow-accumulation analysis (QGIS/GRASS `r.watershed`) or from the OSM waterway network's implied connectivity (snap endpoints within a small tolerance).

J. CITIZEN + REAL-TIME FEATURES

Citizen flood reports

Simple form: GPS (auto via browser `navigator.geolocation`), photo upload, timestamp (auto), estimated depth (slider: ankle/knee/waist/above-waist — easier for citizens than "cm"), free-text description.

Store in `citizen_reports` collection; show as clustered markers; allow NMC-side "verify" action that promotes a report into influencing nearby drains' `risk_score`.

IoT sensors (simulate for demo)




```
javascript
```

// Simple mock sensor emitter (Node.js) — run as a background script during demo

```
setInterval(async () => {  
  const reading = {  
    sensor_id: "WL-042",  
    type: "water_level",  
    value: (Math.random() * 100).toFixed(1),  
    timestamp: new Date()  
  };  
  await fetch("http://localhost:5000/api/sensors/ingest", {  
    method: "POST", headers: {"Content-Type": "application/json"},  
    body: JSON.stringify(reading)  
  });  
}, 5000);
```

Frame this honestly on stage as: "here is a working simulated sensor pipeline; the same ingestion API works with real LoRaWAN/NB-IoT water-level sensors when NMC deploys them — we designed for that from day one."

Flood-aware routing

Simplest MVP: maintain a `blocked_road_segments` list (manually or from citizen reports + flood_zones intersection); when a user requests a route, call an OSRM instance (self-hosted or public demo server) with those road segments' weight set to `Infinity`/excluded, forcing an alternate path. Show both the "normal" and "flood-safe" route on Leaflet with different colors, and label avoided segments in red with a small "🚧 flooded" icon.

K. CIVICPROOF — BLOCKCHAIN LAYER (Algorand)

Why Algorand fits this specific use case

Instant finality, very low & predictable transaction fees — good for frequent, small civic records (thousands of maintenance events) without cost blowing up.

Algorand Standard Assets (ASA) could represent each "work order" as a tokenized record if you want to go further than plain hashing.

Note field on transactions is enough for simple hash-anchoring — you don't need a custom smart contract for MVP.

Algorand's PPoS (Pure Proof-of-Stake) network has been positioned by the Algorand Foundation itself for government/public-infrastructure use cases (data integrity, asset tokenization, financial oversight) — reasonable to cite as directional validation, without overclaiming specific government adoption in India.

On-chain (minimal, MVP-appropriate)



```
"work_order_id": "WO-2026-0417",
"drain_id": "D042",
"event_type": "desilting_completed",
"timestamp": "2026-08-11T10:00:00Z",
"record_hash": "sha256:9f86d081884c7d659a2f...",
"verified_by": "NMC-PHE-officer-id-hash" // hash, not raw PII
```

Store this as the **note field** of a zero-amount Algorand payment transaction from a project-controlled account, OR mint a tiny ASA per work order if you want an explicit on-chain "certificate" object for the demo (more visually impressive for judges — "here's the NFT-like certificate for this drain cleaning").

Off-chain (everything heavy or private)

Before/after photos → store in cloud storage (S3-compatible or even local `/uploads` for demo) → only the **hash** goes on-chain

GIS files, CCTV, large sensor time-series → MongoDB / file storage

Citizen personal information → **never on-chain**, stored securely off-chain with access control

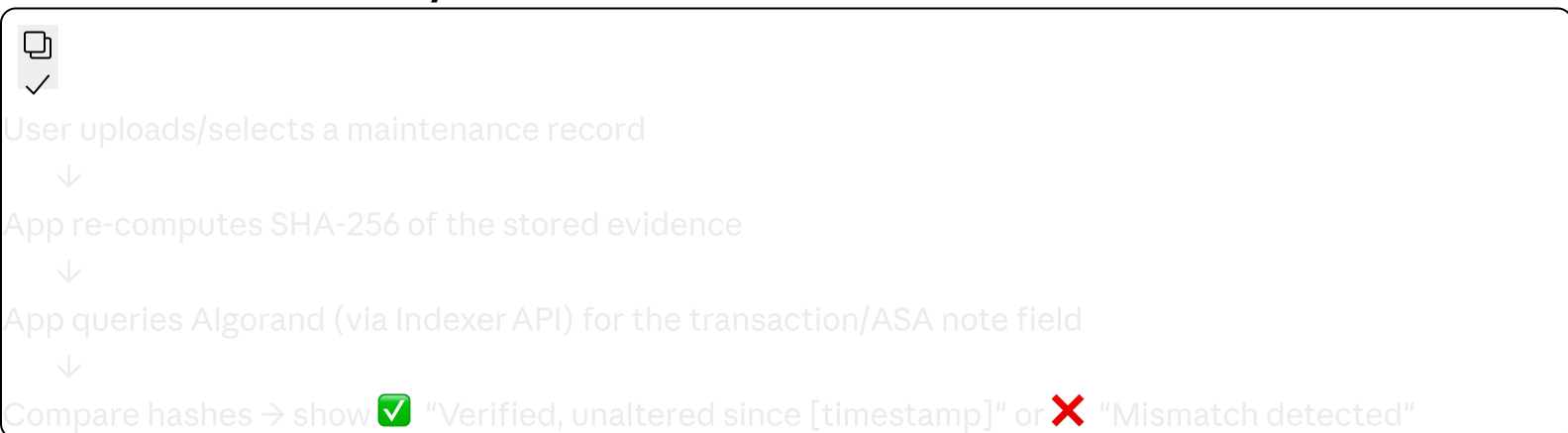
How the hash proves tamper-evidence

Compute `SHA-256(photo_bytes + metadata_json)` off-chain

Anchor that hash on Algorand (immutable, timestamped, publicly queryable via any Algorand block explorer/indexer)

Later, anyone (citizen, auditor, journalist) can re-hash the same evidence file and compare it to the on-chain hash — if they match, the record has not been altered since anchoring. If someone tries to backdate or swap the "after" photo, the hash won't match → tamper detected.

Verification flow (what your demo shows)



Smart-contract-based work-order/payment flow (optional, stretch goal)

A PyTeal/Beaker (or TEALScript, per Algorand's current TS SDK) smart contract could:

Escrow a payment when a work order is created

Release payment only when both an "official approval" transaction AND the after-photo hash are submitted

This demonstrates genuine "smart contract" logic beyond simple hashing, but **keep it optional** — a working hash-anchoring MVP is more valuable to demo reliably than a half-working payment contract under hackathon time pressure.

HACKATHON MVP RECOMMENDATION (what to actually build in the time you have)

Given typical hackathon time budgets (24–48 hrs), prioritize in this order:

Tier 1 — Must build (this alone is a strong, complete demo):

Leaflet map of Nagpur with real OSM waterways + your synthetic drain network overlay

MongoDB + Express API with bbox-filtered endpoints

Flood-risk scoring per drain (using the VNIT FR-model-inspired weighted formula), color-coded on map

Click-a-drain → metadata popup with the "why is this risky" breakdown

A flood-zones layer replicating (or citing + approximating) the VNIT Nagpur susceptibility classes

Citizen flood-report form + map markers

Tier 2 — Build if Tier 1 is done early (this differentiates you): 7. Drainage graph (NetworkX) with a working "what floods if this drain fails" query 8. CivicProof hash-anchoring MVP on Algorand TestNet with a simple verify button 9. Simulated IoT sensor feed

Tier 3 — Stretch / mention as roadmap only: 10. Flood-aware routing (OSRM integration) 11. Smart-contract payment/work-order escrow 12. Vector-tile pipeline for production-scale performance

Pitch narrative arc (recommended): "Nagpur floods every year (2023: 4 deaths, 10,000 homes) and the government has approved ₹470+ crore in drainage fixes since — but implementation has stalled (only ₹86.63 crore released, 1 of 5 pumping stations built). There is no public drainage GIS for citizens or even planners to see this. We built JalRaksha Nagpur: (1) a real, open, hyperlocal flood-risk map assembled from the only Nagpur-specific peer-reviewed flood model that exists plus live open satellite/rainfall data, (2) a citizen-reporting loop that grows the dataset NMC doesn't publish, and (3) CivicProof — a tamper-evident public ledger so citizens can verify whether that ₹204 crore of nullah-desilting work actually happened, not just was announced."

TOP 10 RESOURCES TO GO GET RIGHT NOW

NMC Disaster Management Plan 2022-23 PDF —

https://www.nmcnagpur.gov.in/assets/250/2022/07/mediafiles/Aapatti_Vyavsthapan_Arakhada2022.pdf

Gaurkhede & Adane (2023) flood susceptibility paper (full methodology + parameter tables) —

https://aloki.hu/pdf/2103_23412361.pdf

OSM waterways/drains export for Nagpur bbox via Overpass Turbo — <https://overpass-turbo.eu>

SRTM DEM for Nagpur — <https://earthexplorer.usgs.gov> or <https://opentopography.org>

Bhuvan LULC/flood-hazard WMS layers —

https://bhuvan.nrsc.gov.in/wiki/index.php/How_to_use_WMS_services

NEERI Environmental Status Report, Nagpur —

https://www.nmcnagpur.gov.in/assets/250/2021/06/mediafiles/Final_2020_ESR_-_AK.pdf

DDMA Nagpur District Disaster Management Plan (rainfall baseline) —

<https://static.s3waas.gov.in/s3d1f491a404d6854880943e5c3cd9ca25/uploads/2018/03/2018031651.pdf>

RMC Nagpur (IMD) rainfall data page — https://www.imdnagpur.gov.in/pages/rainfall_drms.php

India-WRIS rainfall/river dashboards — <https://indiawris.gov.in>

MRSAC Geoportal (Nagpur-based state RS/GIS body) —

https://mrsac.gov.in/MRSAC/pages/web_applications

NEXT MESSAGE TO SEND TO ANY LLM TO RESUME WORK

Paste this file plus one line like:

"I'm at Phase X, step Y in the tracker. Help me with [specific next task]."

That's all the context any assistant will need to pick up exactly where you left off.